

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 8: Deploying a Regression Model on Raspberry Pi with Live Sensor Data

Name of Student: Bushra Nazir

Registration No.: 1071-F22

Date of Experiment: 7-5-2026

Submitted To: Engr.Asad

Objectives

By the end of this experiment, students will be able to:

- Explain the difference between training a regression model and deploying it for inference.
- Collect a structured dataset from a physical sensor (DHT11) connected to Raspberry Pi and save it to a CSV file.
- Train a scikit-learn Linear Regression model on the collected sensor data.
- Evaluate model performance using R^2 Score and Mean Squared Error (MSE).
- Deploy the trained model on Raspberry Pi to make live predictions on new sensor readings.
- Display actual vs. predicted values side-by-side on an SSD1306 OLED display.

Background Theory

The Real-World Problem

Imagine you are an engineer at a smart agriculture company in Lahore. You have hundreds of low-cost IoT sensor nodes deployed across a farm. Each node has a temperature and humidity sensor but sending all raw data to the cloud is expensive it uses bandwidth, requires constant internet, and introduces latency. The question becomes: can the device itself be intelligent enough to make decisions locally?

This idea running machine learning directly on embedded or edge devices is called Edge AI or TinyML. Companies like Google (with Coral/Edge TPU), ARM (with Ethos NPU), and STMicroelectronics are investing billions into this space. Raspberry Pi, with its full Linux OS and Python environment, is the perfect starting point to understand this concept before moving to more constrained hardware.

Regression is one of the most fundamental and widely deployed ML techniques in industry. It answers the question: given what we know (input features), what value can we predict (output)?

In sensor-based systems, regression is used everywhere:

- Predicting remaining battery life from voltage readings.
- Estimating soil moisture from capacitance sensor values.

- Predicting engine temperature from RPM and load.
- Forecasting electricity demand from historical usage and weather data.

In this lab, we use the physical relationship between humidity and temperature as humidity rises in a closed environment, temperature typically follows a pattern. We train a model to learn this relationship from real sensor readings, then use it to predict temperature from humidity alone.

The Full Machine Learning Pipeline on an Edge Device

A key insight this lab teaches is that machine learning has two distinct phases. This experiment covers both phases on the same physical device:

Phase	What Happens	Where It Runs
1 Data Collection	Sensor readings logged to CSV file over time	Raspberry Pi
2 Training	sklearn model fitted on collected CSV data	Raspberry Pi
3 Evaluation	R ² and MSE computed on test split	Raspberry Pi
4 Inference / Deployment	Model predicts from new live sensor readings	Raspberry Pi
5 Display	Actual vs. predicted shown on OLED	Raspberry Pi

Linear Regression Review

Linear regression models the relationship between a dependent variable Y and one or more independent variables X by fitting a straight line to the data. For simple (single-feature) linear regression:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where β_0 is the intercept, β_1 is the slope (coefficient), and ϵ is the error term. The algorithm finds β_0 and β_1 by minimizing the sum of squared errors between predicted and actual values (Ordinary Least Squares).

In our sensor experiment:

- X = Humidity (%) measured by DHT11
- Y = Temperature (°C) measured by DHT11
- The model learns: Temperature = $\beta_0 + \beta_1 \times \text{Humidity}$

Model Evaluation Metrics

After training, we measure how well the model fits the data using two key metrics:

Metric	Formula / Meaning	Ideal Value
MeanSquared Error (MSE)	Average of squared differences between actual and predicted values. Lower is better.	Close to 0
R ² Score (Coefficient of Determination)	Proportion of variance in Y explained by X. Ranges from 0 to 1.	Close to 1.0

An R^2 of 0.85, for example, means the model explains 85% of the variation in temperature using humidity as the predictor. For DHT11 sensor data in a relatively stable indoor environment, you can typically expect R^2 between 0.6 and 0.9 depending on environmental conditions.

What Makes This "Edge ML"?

Traditional ML workflows look like this: data collected on a device → uploaded to cloud server → model trained on server → predictions returned to device. This has three problems: latency (round-trip time), cost (bandwidth and cloud compute), and dependency (no internet = no intelligence).

Edge ML moves the intelligence to the device itself. In this lab, the Raspberry Pi performs all five stages listed earlier locally no cloud, no internet required during inference. This is the architecture used in real deployed IoT systems.

scikit-learn on Raspberry Pi

scikit-learn is a full Python machine learning library that runs natively on Raspberry Pi OS (Debian-based Linux). Its LinearRegression class uses NumPy under the hood and is efficient enough to train on hundreds of sensor readings in milliseconds on the Pi's ARM processor. This makes it ideal for prototyping edge ML applications before potentially moving to optimized runtimes like TensorFlow Lite or ONNX Runtime for more constrained hardware.

4. Equipment Required

Component	Specification / Notes	Quantity
Raspberry Pi	Model 3B, 3B+, 4B, or Zero 2W with Raspberry Pi OS	1
DHT11 Sensor	Temperature & Humidity sensor module (with built-in pull-up resistor preferred)	1
OLED Display	0.96" SSD1306, I2C interface, 128×64 pixels	1
Bread board	Half-size or full-size	1 8–
Jumper Wires	Male-to-Female (Pi GPIO to breadboard)	10 1
USB PowerSupply	5V / 2.5A minimum for Pi	1
MicroSD Card	16GB+ with Raspberry Pi OS installed	1
Monitor / SSHAccess	HDMI monitor OR laptop with SSH/VNC for headless operation	

Circuit Connections

DHT11 Pin	Connect To (Raspberry Pi)	Physical Pin No.
VCC	3.3V Power Rail	Pin 1
DATA / OUT	GPIO4	Pin 7
GND	Ground	Pin 6

OLED Display (SSD1306) Connections

OLED Pin	Connect To (Raspberry Pi)	Physical Pin No.
VCC	3.3V Power Rail	Pin 1
GND	Ground	Pin 9
SCL	GPIO3 (I2C Clock)	Pin 5
SDA	GPIO2 (I2C Data)	Pin 3

Enable I2C on Raspberry Pi

I2C must be enabled before the OLED will communicate. Run the following in terminal:

```
sudo raspi-config
```

Navigate to: Interface Options → I2C → Enable → Yes → Finish. Then reboot.

Verify the OLED is detected on the I2C bus:

```
sudo i2cdetect -y 1
```

You should see address 0x3C (or 0x3D) appear in the grid output. This confirms the OLED is wired correctly and I2C is active.

Software Setup & Library Installation

Open a terminal on the Raspberry Pi and run the following commands in order. An internet connection is required for this step only once libraries are installed, the experiment runs fully offline.

Update the System

```
sudo apt update && sudo apt upgrade -y
```

Install Python Libraries

```
pip install adafruit-blinka
pip install adafruit-circuitpython-dht
pip install adafruit-circuitpython-ssd1306
pip install scikit-learn
pip install pillow
```

Install libgpod (Required for DHT on newer Pi OS)

```
sudo apt install python3-libgpod -y
```

Verify Installation

```
python3 -c "import sklearn; import adafruit_dht; import adafruit_ssd1306; print('All OK')"
```

If you see "All OK" proceed to the experiment. If any import fails, re-run the corresponding pip install command.

Procedure

This experiment is divided into three phases. Follow each phase completely before

moving to the next.

Phase 1 Data Collection Script

Create a new file named phase1_collect.py on the Raspberry Pi:

```
nano phase1_collect.py
```

Type or paste the following code:

```
import time
import csv
import board
import adafruit_dht

# --- Configuration ---
SENSOR_PIN    = board.D4          # GPIO4, Physical Pin 7
OUTPUT_FILE   = 'sensor_data.csv'
SAMPLE_COUNT  = 60                # Number of readings to collect
INTERVAL_SECS = 10                # Seconds between readings

# --- Initialize DHT11 ---
dht = adafruit_dht.DHT11(SENSOR_PIN)
print(f'Collecting {SAMPLE_COUNT} readings. This will take approx. '),
print(f'{SAMPLE_COUNT * INTERVAL_SECS / 60:.1f} minutes. Do not interrupt.')
print('Reading | Humidity (%) | Temperature (C)')
print('-' * 48)
```

```

with open(OUTPUT_FILE, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['reading', 'humidity', 'temperature'])
    count = 0
    while count < SAMPLE_COUNT:
        try:
            humidity = dht.humidity
            temperature = dht.temperature
            if humidity is not None and temperature is not None:
                count += 1
                writer.writerow([count, humidity, temperature])
                f.flush() # Write immediately to disk
                print(f' {count:3d}      |    {humidity:6.1f}      |    {temp
erature:6.1f}')
            else:
                print(' [Read failed - retrying...]\n')
        except RuntimeError as e:
            print(f' [Sensor error: {e}]\n')
            time.sleep(INTERVAL_SECS)

dht.exit()
print(f'\nDone! Data saved to {OUTPUT_FILE}')

```

Save the file (Ctrl+O, Enter, Ctrl+X in nano) and run it:

```
python3 phase1_collect.py
```

Wait for all 60 readings to complete. You will see a live table printed in the terminal. The data is saved to sensor_data.csv in the same directory. While it runs, observe the values humidity and temperature should both vary slightly over time as the environment changes naturally.

Phase 2 Model Training & Evaluation Script

Create a new file named phase2_train.py:

```
nano phase2_train.py
```

```

import pandas as pd
import numpy as np
import pickle
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# --- Load the CSV collected in Phase 1 ---
df = pd.read_csv('sensor_data.csv')
print(f'Dataset loaded: {len(df)} readings')
print(df.head())

# --- Features and Target ---
X = df[['humidity']].values      # Input: humidity (2D array for sklearn)
y = df['temperature'].values     # Output: temperature

# --- Train / Test Split (80% train, 20% test) ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print(f'\nTraining samples : {len(X_train)}')
print(f'Testing samples : {len(X_test)}')

#--- Train the Model ---
model = LinearRegression()
model.fit(X_train, y_train)

print(f'\nModel Trained!')
print(f' Intercept   (b0) : {model.intercept_:.4f}')
print(f' Coefficient(b1) : {model.coef_[0]:.4f}')
print(f' Equation: Temp = {model.intercept_:.2f} + {model.coef_[0]:.4f}x
Humidity')

# --- Evaluate ---
y_pred = model.predict(X_test)
mse     = mean_squared_error(y_test, y_pred)
r2      = r2_score(y_test, y_pred)

print(f'\nModel Evaluation on Test Set:')
print(f' Mean Squared Error (MSE) : {mse:.4f}')
print(f' R-Squared (R2) Score      : {r2:.4f}')

```

```
# --- Show actual vs predicted for testset ---
print('\nActual vs Predicted (Test Set):')
print(' Actual Temp   | Predicted Temp   | Error')
print(' ' + '-' * 42)
for actual, pred in zip(y_test, y_pred):
    error = actual - pred
    print(f'   {actual:7.2f}C   |   {pred:7.2f}C   |   {error:+.2f}')

# --- Save the trained model to disk ---
with open('regression_model.pkl', 'wb') as f:
    pickle.dump(model, f)
print('\nModel saved as regression_model.pkl')
print('Proceed to Phase 3 for live deployment.')
```

Run the training script:

```
python3 phase2_train.py
```

Read the terminal output carefully. Note the equation printed (e.g., $\text{Temp} = 18.42 + 0.183 \times \text{Humidity}$), the R^2 score, and the actual vs predicted comparison table. Record these in your lab report.

Phase 3 Live Inference & OLED Display Script

Create a new file named `phase3_deploy.py`:

```
import time
import pickle
import board
import busio
import adafruit_dht
import adafruit_ssd1306
from PIL import Image, ImageDraw, ImageFont

# --- Load Trained Model ---
with open('regression_model.pkl', 'rb') as f:
    model = pickle.load(f)
print('Model loaded. Starting live inference...')
print('Press Ctrl+C to stop.')

# --- Initialize DHT11 ---
dht = adafruit_dht.DHT11(board.D4)

# --- Initialize OLED via I2C ---
i2c = busio.I2C(board.SCL, board.SDA)
oled = adafruit_ssd1306.SSD1306_I2C(128, 64, i2c)
oled.fill(0)
oled.show()
```

```

def display_results(humidity, actual_temp, predicted_temp):
    """Render inference results on OLED display."""
    img = Image.new('1', (128, 64), 0)
    draw = ImageDraw.Draw(img)
    font = ImageFont.load_default()

    # Header
    draw.text((0, 0), 'EdgeML:Regression', font=font, fill=1)
    draw.line([(0, 10), (127, 10)], fill=1)

    # Data rows
    draw.text((0, 13), f'Humidity      :{humidity:.1f}%', font=font,
fill=1)
    draw.text((0, 24), f'Actual        :{actual_temp:.1f} C', font=font,
fill=1)
    draw.text((0, 35), f'Predicted    :{predicted_temp:.1f}C', font=font,
fill=1)

    # Error
    error = actual_temp - predicted_temp
    draw.text((0, 46), f'Error          :{error:+.2f}C', font=font,
fill=1)
    draw.line([(0, 57), (127, 57)], fill=1)
    draw.text((0, 58), 'IIUIEdge    AIDemo', font=font,
fill=1)

    oled.image(img)
    oled.show()

# --- Main Inference Loop ---
reading_no = 0
while True:
    try:
        humidity    = dht.humidity
        temperature = dht.temperature

        if humidity is not None and temperature is not None:
            reading_no += 1
            # Inference: feed humidity to trained model
            predicted = model.predict([[humidity]])[0]

        # Show on OLED
        display_results(humidity, temperature, predicted)

        # Also print to terminal
        print(f'Reading{reading_no:3d}|Hum:{humidity:.1f}% | '
              f'Actual:{temperature:.1f}C|Predicted:{predicted:.1f}C
| '

```

```

        else:
            print('[Read failed - retrying...]\n')

    except RuntimeError as e:
        print(f'[Sensor error: {e}]\n')

    time.sleep(5)

# Cleanup (reached only if loop is modified to stop)
dht.exit()
oled.fill(0)
oled.show()

```

Run the deployment script:

```
python3 phase3_deploy.py
```

The OLED display will update every 5 seconds showing: current humidity, actual temperature from the sensor, and the temperature predicted by the trained regression model. The difference between actual and predicted (the error) is displayed in real time. Press Ctrl+C to stop.

Observations & Results

Phase2ModelOutput RecordHere

Parameter	Your Recorded Value
Total readings in dataset	60
Training samples used	12
Test samples used	48
Intercept (b0)	18.5
Coefficient / Slope (b1)	18.5
Regression equation	Temp = 18.5 + 0.21(Humidity)
Mean Squared Error (MSE)	0.85
R ² Score	0.72

Phase 3 Live Inference Record 10 Readings

Reading #	Humidity (%)	Actual Temp (°C)	Predicted Temp
1	48	28.6	28.5
2	49	28.9	28.8
3	50	29	29
4	51	29.3	29.2
5	52	29.4	29.4
6	53	29.8	29.6
7	54	29.9	29.8
8	55	30.1	30
9	56	30.3	30.3
10	57	30.6	30.5

OLED Display Observation

Sketch or describe what appears on the OLED display during Phase 3:

EXERCISE

Analysis Questions

Answer the following questions in your lab report based on your observations:

1. What does your R^2 score indicate about the relationship between humidity and temperature in your lab environment? If R^2 is low (below 0.5), what could be the reason?
2. During Phase 3, are the errors (actual minus predicted) mostly positive or negative? What does the sign of the error tell you?
3. Explain in your own words the difference between Phase 2 (training) and Phase 3 (inference). Why must these be separate phases?
4. The model was saved as a .pkl file using pickle. Why is saving the model important for an Edge AI deployment? What would happen if we skipped this step?
5. If you wanted to improve model accuracy, what changes would you make to the data collection phase? List at least three.
6. Define a variable `TEMP_THRESHOLD = 32` at the top of the script.
 - In `display_results()`, add an if/else for the bottom line of the OLED: show 'WARNING!' if `predicted >= TEMP_THRESHOLD`, otherwise show the error value.
 - In the terminal print statement, add a visual marker like `>>> ALERT <<<` when the threshold is exceeded.
 - Test by gently warming the DHT11 sensor with your hand to push humidity and temperature higher until the prediction crosses the threshold.

1. What does your R^2 score indicate about the relationship between humidity and temperature? If R^2 is low (< 0.5), what could be the reason?

R^2 shows how well humidity can explain or predict temperature.

A high R^2 means a strong relationship.

A low R^2 means humidity does not explain temperature well.

Low R^2 may be caused by limited data, sensor noise, or other factors affecting temperature.

2. During Phase 3, are the errors (actual - predicted) mostly positive or negative? What does the sign tell you?

Positive error: the model predicted a value lower than the actual temperature (underestimation).

Negative error: the model predicted a value higher than the actual temperature (overestimation).

The sign indicates whether the model is predicting too low or too high.

3. Difference between Phase 2 (training) and Phase 3 (inference). Why must they be separate?

Training: The model learns patterns from collected data.

Inference: The trained model makes predictions on new data.

They must be separate to test whether the model can accurately predict unseen data.

4. Why is saving the model as a .pkl file important?

It allows the trained model to be reused without retraining.

It saves time and computing resources.

If skipped, the model would need to be trained again every time the program runs.

5. How could you improve model accuracy?

Collect more data samples.

Gather data under different environmental conditions.

Remove incorrect or noisy measurements.

6. TEMP_THRESHOLD = 32 Modification

If predicted temperature is $\geq 32^\circ\text{C}$, display "WARNING!" on the OLED and show ">>> ALERT <<<" in the terminal.

Otherwise, display the prediction error on the OLED.

Warming the DHT11 sensor with your hand should increase the readings and help test the warning condition.

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 8

Deploying a Regression Model on Raspberry Pi with Live Sensor Data

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	<i>Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:</i>				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	<i>Use of tools, equipment and materials with:</i>				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	<i>Degree of reminders to follow safety procedure with:</i>				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	<i>Data record, Analysis and Evaluation or Designed Algorithm is:</i>				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	<i>Procedural awareness of the completed task is:</i>				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	<i>Degree of positive interaction as an individual or within a group:</i>				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) : 10

Marks Obtained (PO + CO + AO) :

Instructor